DATA STRUCTURES

PRACTICAL RECORD – C PROGRAMS WITH SAMPLE OUTPUTS

21 separate programs covering all 14 practical questions

Includes aim, complete C source code and sample output for each program.

Preparation copy

Practical Program Index

1(a). Stack ADT using Array

1(b). Linear Queue ADT using Array

1(c). Circular Queue ADT using Array

2. Singly Linked List

3(a). Stack ADT using Linked List

3(b). Linear Queue ADT using Linked List

4. Polynomial Manipulation using Linked List

5(a). Evaluation of Postfix Expression

5(b). Infix to Postfix Conversion

6. Binary Search Tree

7. AVL Tree

8. Heap using Priority Queue

9. Dijkstra's Algorithm

10. Prim's Algorithm

11(a). Linear Search

11(b). Binary Search

12(a). Insertion Sort

12(b). Selection Sort

13. Merge Sort

14(a). Hashing using Linear Probing

14(b). Hashing using Quadratic Probing

1(a). Stack ADT using Array

Aim: Implement push, pop and display operations on a stack using an array.

Program:

```
#include <stdio.h>
#define MAX 5

int stack[MAX], top = -1;

void push(int value) {
    if (top == MAX - 1) {
        printf("Stack Overflow\n");
        return;
    }
    stack[++top] = value;
    printf("%d pushed\n", value);
}

void pop(void) {
    if (top == -1) {
        printf("Stack Underflow\n");
        return;
    }
    printf("Popped: %d\n", stack[top--]);
}

void display(void) {
    int i;
    if (top == -1) {
        printf("Stack is empty\n");
        return;
    }
    printf("Stack: ");
    for (i = top; i >= 0; i--)
        printf("%d ", stack[i]);
    printf("\n");
}

int main(void) {
    push(10);
    push(20);
    push(30);
    display();
    pop();
    display();
    return 0;
}
```

Sample Output:

```
10 pushed
20 pushed
30 pushed
Stack: 30 20 10
Popped: 30
Stack: 20 10
```

1(b). Linear Queue ADT using Array

Aim: Implement enqueue, dequeue and display operations using an array.

Program:

```
#include <stdio.h>
#define MAX 5

int queue[MAX], front = 0, rear = -1;

void enqueue(int value) {
    if (rear == MAX - 1) {
        printf("Queue Overflow\n");
        return;
    }
    queue[++rear] = value;
    printf("%d inserted\n", value);
}

void dequeue(void) {
    if (front > rear) {
        printf("Queue Underflow\n");
        return;
    }
    printf("Deleted: %d\n", queue[front++]);
}

void display(void) {
    int i;
    if (front > rear) {
        printf("Queue is empty\n");
        return;
    }
    printf("Queue: ");
    for (i = front; i <= rear; i++)
        printf("%d ", queue[i]);
    printf("\n");
}

int main(void) {
    enqueue(10);
    enqueue(20);
    enqueue(30);
    display();
    dequeue();
    display();
    return 0;
}
```

Sample Output:

```
10 inserted
20 inserted
30 inserted
Queue: 10 20 30
Deleted: 10
Queue: 20 30
```

1(c). Circular Queue ADT using Array

Aim: Implement insertion, deletion and display in a circular queue.

Program:

```
#include <stdio.h>
#define MAX 5

int queue[MAX], front = -1, rear = -1;

void enqueue(int value) {
    if ((rear + 1) % MAX == front) {
        printf("Queue Overflow\n");
        return;
    }
    if (front == -1)
        front = 0;
    rear = (rear + 1) % MAX;
    queue[rear] = value;
    printf("%d inserted\n", value);
}

void dequeue(void) {
    if (front == -1) {
        printf("Queue Underflow\n");
        return;
    }
    printf("Deleted: %d\n", queue[front]);
    if (front == rear) {
        front = rear = -1;
    } else {
        front = (front + 1) % MAX;
    }
}

void display(void) {
    int i;
    if (front == -1) {
        printf("Queue is empty\n");
        return;
    }
    printf("Queue: ");
    i = front;
    while (1) {
        printf("%d ", queue[i]);
        if (i == rear)
            break;
        i = (i + 1) % MAX;
    }
    printf("\n");
}

int main(void) {
    enqueue(10);
    enqueue(20);
    enqueue(30);
    display();
    dequeue();
    enqueue(40);
    display();
    return 0;
}
```

Sample Output:

```
10 inserted
20 inserted
30 inserted
Queue: 10 20 30
Deleted: 10
40 inserted
Queue: 20 30 40
```

2. Singly Linked List

Aim: Create a singly linked list, insert nodes at the end and display the list.

Program:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *head = NULL;

void insertEnd(int value) {
    struct Node *newNode = malloc(sizeof(struct Node));
    struct Node *temp;
    if (newNode == NULL) {
        printf("Memory allocation failed\n");
        return;
    }
    newNode->data = value;
    newNode->next = NULL;
    if (head == NULL) {
        head = newNode;
        return;
    }
    temp = head;
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = newNode;
}

void display(void) {
    struct Node *temp = head;
    printf("List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main(void) {
    insertEnd(10);
    insertEnd(20);
    insertEnd(30);
    display();
    return 0;
}
```

Sample Output:

```
List: 10 -> 20 -> 30 -> NULL
```

3(a). Stack ADT using Linked List

Aim: Implement push, pop and display operations using a linked list.

Program:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};
struct Node *top = NULL;

void push(int value) {
    struct Node *newNode = malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed\n");
        return;
    }
    newNode->data = value;
    newNode->next = top;
    top = newNode;
    printf("%d pushed\n", value);
}

void pop(void) {
    struct Node *temp;
    if (top == NULL) {
        printf("Stack Underflow\n");
        return;
    }
    temp = top;
    printf("Popped: %d\n", temp->data);
    top = top->next;
    free(temp);
}

void display(void) {
    struct Node *temp = top;
    printf("Stack: ");
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main(void) {
    push(10);
    push(20);
    push(30);
    display();
    pop();
    display();
    return 0;
}
```

Sample Output:

```
10 pushed
20 pushed
30 pushed
Stack: 30 20 10
Popped: 30
Stack: 20 10
```

3(b). Linear Queue ADT using Linked List

Aim: Implement enqueue, dequeue and display operations using a linked list.

Program:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};
struct Node *front = NULL, *rear = NULL;

void enqueue(int value) {
    struct Node *newNode = malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed\n");
        return;
    }
    newNode->data = value;
    newNode->next = NULL;
    if (rear == NULL) {
        front = rear = newNode;
    } else {
        rear->next = newNode;
        rear = newNode;
    }
    printf("%d inserted\n", value);
}

void dequeue(void) {
    struct Node *temp;
    if (front == NULL) {
        printf("Queue Underflow\n");
        return;
    }
    temp = front;
    printf("Deleted: %d\n", temp->data);
    front = front->next;
    if (front == NULL)
        rear = NULL;
    free(temp);
}

void display(void) {
    struct Node *temp = front;
    printf("Queue: ");
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main(void) {
    enqueue(10);
    enqueue(20);
    enqueue(30);
    display();
    dequeue();
    display();
    return 0;
}
```

Sample Output:

```
10 inserted
20 inserted
30 inserted
Queue: 10 20 30
Deleted: 10
Queue: 20 30
```

4. Polynomial Manipulation using Linked List

Aim: Represent two polynomials using linked lists and add them.

Program:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int coeff, exp;
    struct Node *next;
};

struct Node *insert(struct Node *head, int c, int e) {
    struct Node *n = malloc(sizeof(struct Node));
    struct Node *t;
    if (n == NULL)
        return head;
    n->coeff = c;
    n->exp = e;
    n->next = NULL;
    if (head == NULL)
        return n;
    t = head;
    while (t->next != NULL)
        t = t->next;
    t->next = n;
    return head;
}

struct Node *add(struct Node *a, struct Node *b) {
    struct Node *result = NULL;
    while (a != NULL && b != NULL) {
        if (a->exp == b->exp) {
            result = insert(result, a->coeff + b->coeff, a->exp);
            a = a->next;
            b = b->next;
        } else if (a->exp > b->exp) {
            result = insert(result, a->coeff, a->exp);
            a = a->next;
        } else {
            result = insert(result, b->coeff, b->exp);
            b = b->next;
        }
    }
    while (a != NULL) {
        result = insert(result, a->coeff, a->exp);
        a = a->next;
    }
    while (b != NULL) {
        result = insert(result, b->coeff, b->exp);
        b = b->next;
    }
    return result;
}

void display(struct Node *p) {
    while (p != NULL) {
        printf("%dx^%d", p->coeff, p->exp);
        p = p->next;
        if (p != NULL)
            printf(" + ");
    }
    printf("\n");
}

int main(void) {
    struct Node *p1 = NULL, *p2 = NULL, *sum;
    p1 = insert(p1, 5, 2);
    p1 = insert(p1, 4, 1);
    p1 = insert(p1, 2, 0);
    p2 = insert(p2, 3, 2);
    p2 = insert(p2, 2, 1);
    p2 = insert(p2, 1, 0);

    printf("First polynomial: ");
    display(p1);
    printf("Second polynomial: ");
    display(p2);
```

```
    sum = add(p1, p2);
    printf("Sum: ");
    display(sum);
    return 0;
}
```

Sample Output:

```
First polynomial: 5x^2 + 4x^1 + 2x^0
Second polynomial: 3x^2 + 2x^1 + 1x^0
Sum: 8x^2 + 6x^1 + 3x^0
```

5(a). Evaluation of Postfix Expression

Aim: Evaluate a postfix expression containing single-digit operands and the operators +, -, * and /.

Program:

```
#include <stdio.h>
#include <ctype.h>

int stack[50], top = -1;

void push(int value) {
    stack[++top] = value;
}

int pop(void) {
    return stack[top--];
}

int main(void) {
    char exp[50];
    int i, a, b, result;
    printf("Enter postfix expression: ");
    scanf("%49s", exp);

    for (i = 0; exp[i] != '\0'; i++) {
        if (isdigit((unsigned char)exp[i])) {
            push(exp[i] - '0');
        } else {
            b = pop();
            a = pop();
            switch (exp[i]) {
                case '+': result = a + b; break;
                case '-': result = a - b; break;
                case '*': result = a * b; break;
                case '/':
                    if (b == 0) {
                        printf("Division by zero\n");
                        return 1;
                    }
                    result = a / b;
                    break;
                default:
                    printf("Invalid operator\n");
                    return 1;
            }
            push(result);
        }
    }
    if (top == 0)
        printf("Result: %d\n", pop());
    else
        printf("Invalid expression\n");
    return 0;
}
```

Sample Output:

```
Enter postfix expression: 23*54*+
Result: 26
```

5(b). Infix to Postfix Conversion

Aim: Convert an infix expression with single-character operands to postfix form.

Program:

```
#include <stdio.h>
#include <ctype.h>

char stack[50];
int top = -1;

int precedence(char op) {
    if (op == '^') return 3;
    if (op == '*' || op == '/') return 2;
    if (op == '+' || op == '-') return 1;
    return 0;
}

int main(void) {
    char infix[50], postfix[50];
    int i, k = 0;
    char ch;
    printf("Enter infix expression: ");
    scanf("%49s", infix);

    for (i = 0; infix[i] != '\0'; i++) {
        ch = infix[i];
        if (isalnum((unsigned char)ch)) {
            postfix[k++] = ch;
        } else if (ch == '(') {
            stack[++top] = ch;
        } else if (ch == ')') {
            while (top != -1 && stack[top] != '(')
                postfix[k++] = stack[top--];
            if (top != -1)
                top--;
        } else {
            while (top != -1 && stack[top] != '(' &&
                   (precedence(stack[top]) > precedence(ch) ||
                   (precedence(stack[top]) == precedence(ch) && ch != '^')))
                postfix[k++] = stack[top--];
            stack[++top] = ch;
        }
    }
    while (top != -1)
        postfix[k++] = stack[top--];
    postfix[k] = '\0';

    printf("Postfix expression: %s\n", postfix);
    return 0;
}
```

Sample Output:

```
Enter infix expression: (A+B)*C
Postfix expression: AB+C*
```

6. Binary Search Tree

Aim: Insert values into a binary search tree and display inorder traversal.

Program:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right;
};

struct Node *insert(struct Node *root, int value) {
    if (root == NULL) {
        struct Node *n = malloc(sizeof(struct Node));
        if (n == NULL)
            return NULL;
        n->data = value;
        n->left = n->right = NULL;
        return n;
    }
    if (value < root->data)
        root->left = insert(root->left, value);
    else if (value > root->data)
        root->right = insert(root->right, value);
    return root;
}

void inorder(struct Node *root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

int main(void) {
    struct Node *root = NULL;
    root = insert(root, 50);
    root = insert(root, 30);
    root = insert(root, 70);
    root = insert(root, 20);
    root = insert(root, 40);
    root = insert(root, 60);
    root = insert(root, 80);

    printf("Inorder traversal: ");
    inorder(root);
    printf("\n");
    return 0;
}
```

Sample Output:

```
Inorder traversal: 20 30 40 50 60 70 80
```

7. AVL Tree

Aim: Insert values into an AVL tree and display its preorder traversal.

Program:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int key, height;
    struct Node *left, *right;
};

int max(int a, int b) { return a > b ? a : b; }
int height(struct Node *n) { return n == NULL ? 0 : n->height; }

struct Node *newNode(int key) {
    struct Node *n = malloc(sizeof(struct Node));
    if (n == NULL) return NULL;
    n->key = key;
    n->left = n->right = NULL;
    n->height = 1;
    return n;
}

struct Node *rightRotate(struct Node *y) {
    struct Node *x = y->left, *t2 = x->right;
    x->right = y;
    y->left = t2;
    y->height = 1 + max(height(y->left), height(y->right));
    x->height = 1 + max(height(x->left), height(x->right));
    return x;
}

struct Node *leftRotate(struct Node *x) {
    struct Node *y = x->right, *t2 = y->left;
    y->left = x;
    x->right = t2;
    x->height = 1 + max(height(x->left), height(x->right));
    y->height = 1 + max(height(y->left), height(y->right));
    return y;
}

int balance(struct Node *n) {
    return n == NULL ? 0 : height(n->left) - height(n->right);
}

struct Node *insert(struct Node *node, int key) {
    int b;
    if (node == NULL) return newNode(key);
    if (key < node->key)
        node->left = insert(node->left, key);
    else if (key > node->key)
        node->right = insert(node->right, key);
    else return node;

    node->height = 1 + max(height(node->left), height(node->right));
    b = balance(node);

    if (b > 1 && key < node->left->key) return rightRotate(node);
    if (b < -1 && key > node->right->key) return leftRotate(node);
    if (b > 1 && key > node->left->key) {
        node->left = leftRotate(node->left);
        return rightRotate(node);
    }
    if (b < -1 && key < node->right->key) {
        node->right = rightRotate(node->right);
        return leftRotate(node);
    }
    return node;
}

void preorder(struct Node *root) {
    if (root != NULL) {
        printf("%d ", root->key);
        preorder(root->left);
        preorder(root->right);
    }
}
```

```
int main(void) {
    struct Node *root = NULL;
    root = insert(root, 10);
    root = insert(root, 20);
    root = insert(root, 30);
    root = insert(root, 40);
    root = insert(root, 50);
    root = insert(root, 25);

    printf("Preorder traversal: ");
    preorder(root);
    printf("\n");
    return 0;
}
```

Sample Output:

```
Preorder traversal: 30 20 10 25 40 50
```

8. Heap using Priority Queue

Aim: Implement a max-priority queue using a binary max heap.

Program:

```
#include <stdio.h>
#define MAX 20

int heap[MAX], size = 0;

void insert(int value) {
    int i, parent;
    if (size == MAX) {
        printf("Priority queue is full\n");
        return;
    }
    i = size++;
    while (i > 0) {
        parent = (i - 1) / 2;
        if (heap[parent] >= value) break;
        heap[i] = heap[parent];
        i = parent;
    }
    heap[i] = value;
}

int deleteMax(void) {
    int maxValue, last, i = 0, child;
    if (size == 0) {
        printf("Priority queue is empty\n");
        return -1;
    }
    maxValue = heap[0];
    last = heap[--size];
    while (2 * i + 1 < size) {
        child = 2 * i + 1;
        if (child + 1 < size && heap[child + 1] > heap[child])
            child++;
        if (last >= heap[child]) break;
        heap[i] = heap[child];
        i = child;
    }
    if (size > 0) heap[i] = last;
    return maxValue;
}

void display(void) {
    int i;
    printf("Heap: ");
    for (i = 0; i < size; i++)
        printf("%d ", heap[i]);
    printf("\n");
}

int main(void) {
    insert(30);
    insert(10);
    insert(50);
    insert(20);
    insert(40);
    display();
    printf("Deleted highest priority: %d\n", deleteMax());
    display();
    return 0;
}
```

Sample Output:

```
Heap: 50 40 30 10 20
Deleted highest priority: 50
Heap: 40 20 30 10
```

9. Dijkstra's Algorithm

Aim: Find shortest distances from source vertex 0 in a weighted graph with non-negative edges.

Program:

```
#include <stdio.h>
#define V 5
#define INF 99999

int main(void) {
    int graph[V][V] = {
        {0, 10, 0, 5, 0},
        {10, 0, 1, 2, 0},
        {0, 1, 0, 0, 4},
        {5, 2, 0, 0, 2},
        {0, 0, 4, 2, 0}
    };
    int dist[V], visited[V] = {0};
    int i, count, u = -1, min, v, source = 0;

    for (i = 0; i < V; i++) dist[i] = INF;
    dist[source] = 0;

    for (count = 0; count < V - 1; count++) {
        min = INF;
        u = -1;
        for (i = 0; i < V; i++) {
            if (!visited[i] && dist[i] < min) {
                min = dist[i];
                u = i;
            }
        }
        if (u == -1) break;
        visited[u] = 1;

        for (v = 0; v < V; v++) {
            if (!visited[v] && graph[u][v] != 0 &&
                dist[u] != INF &&
                dist[u] + graph[u][v] < dist[v]) {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }

    printf("Shortest distances from vertex %d:\n", source);
    for (i = 0; i < V; i++)
        printf("To %d = %d\n", i, dist[i]);
    return 0;
}
```

Sample Output:

```
Shortest distances from vertex 0:
To 0 = 0
To 1 = 7
To 2 = 8
To 3 = 5
To 4 = 7
```

10. Prim's Algorithm

Aim: Find the minimum spanning tree of a connected, undirected weighted graph.

Program:

```
#include <stdio.h>
#define V 5
#define INF 99999

int main(void) {
    int graph[V][V] = {
        {0, 2, 0, 6, 0},
        {2, 0, 3, 8, 5},
        {0, 3, 0, 0, 7},
        {6, 8, 0, 0, 9},
        {0, 5, 7, 9, 0}
    };
    int selected[V] = {0};
    int edges = 0, i, j, min, x = -1, y = -1, total = 0;

    selected[0] = 1;
    printf("Edges in minimum spanning tree:\n");

    while (edges < V - 1) {
        min = INF;
        x = y = -1;
        for (i = 0; i < V; i++) {
            if (selected[i]) {
                for (j = 0; j < V; j++) {
                    if (!selected[j] && graph[i][j] != 0 &&
                        graph[i][j] < min) {
                        min = graph[i][j];
                        x = i;
                        y = j;
                    }
                }
            }
        }
        if (x == -1) {
            printf("Graph is not connected\n");
            return 1;
        }
        printf("%d - %d : %d\n", x, y, min);
        total += min;
        selected[y] = 1;
        edges++;
    }
    printf("Total cost: %d\n", total);
    return 0;
}
```

Sample Output:

```
Edges in minimum spanning tree:
0 - 1 : 2
1 - 2 : 3
1 - 4 : 5
0 - 3 : 6
Total cost: 16
```

11(a). Linear Search

Aim: Search for an element by checking each array element sequentially.

Program:

```
#include <stdio.h>

int main(void) {
    int a[] = {10, 20, 30, 40, 50};
    int n = 5, key = 30, i, found = 0;

    for (i = 0; i < n; i++) {
        if (a[i] == key) {
            printf("Element found at position %d\n", i + 1);
            found = 1;
            break;
        }
    }
    if (!found)
        printf("Element not found\n");
    return 0;
}
```

Sample Output:

```
Element found at position 3
```

11(b). Binary Search

Aim: Search for an element in a sorted array using binary search.

Program:

```
#include <stdio.h>

int main(void) {
    int a[] = {10, 20, 30, 40, 50, 60};
    int n = 6, key = 40;
    int low = 0, high = n - 1, mid, found = 0;

    while (low <= high) {
        mid = low + (high - low) / 2;
        if (a[mid] == key) {
            printf("Element found at position %d\n", mid + 1);
            found = 1;
            break;
        } else if (a[mid] < key) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    if (!found)
        printf("Element not found\n");
    return 0;
}
```

Sample Output:

```
Element found at position 4
```

12(a). Insertion Sort

Aim: Sort the array in ascending order using insertion sort.

Program:

```
#include <stdio.h>

int main(void) {
    int a[] = {5, 2, 4, 6, 1, 3};
    int n = 6, i, key, j;

    for (i = 1; i < n; i++) {
        key = a[i];
        j = i - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }

    printf("Sorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

Sample Output:

```
Sorted array: 1 2 3 4 5 6
```

12(b). Selection Sort

Aim: Sort the array in ascending order using selection sort.

Program:

```
#include <stdio.h>

int main(void) {
    int a[] = {64, 25, 12, 22, 11};
    int n = 5, i, j, minIndex, temp;

    for (i = 0; i < n - 1; i++) {
        minIndex = i;
        for (j = i + 1; j < n; j++) {
            if (a[j] < a[minIndex])
                minIndex = j;
        }
        temp = a[i];
        a[i] = a[minIndex];
        a[minIndex] = temp;
    }

    printf("Sorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

Sample Output:

```
Sorted array: 11 12 22 25 64
```

13. Merge Sort

Aim: Sort the array in ascending order using merge sort.

Program:

```
#include <stdio.h>

void merge(int a[], int left, int mid, int right) {
    int temp[50], i = left, j = mid + 1, k = 0, x;
    while (i <= mid && j <= right) {
        if (a[i] <= a[j])
            temp[k++] = a[i++];
        else
            temp[k++] = a[j++];
    }
    while (i <= mid) temp[k++] = a[i++];
    while (j <= right) temp[k++] = a[j++];
    for (x = 0; x < k; x++)
        a[left + x] = temp[x];
}

void mergeSort(int a[], int left, int right) {
    int mid;
    if (left < right) {
        mid = left + (right - left) / 2;
        mergeSort(a, left, mid);
        mergeSort(a, mid + 1, right);
        merge(a, left, mid, right);
    }
}

int main(void) {
    int a[] = {38, 27, 43, 3, 9, 82, 10};
    int n = 7, i;

    mergeSort(a, 0, n - 1);
    printf("Sorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

Sample Output:

```
Sorted array: 3 9 10 27 38 43 82
```

14(a). Hashing using Linear Probing

Aim: Insert keys into a hash table using linear probing to resolve collisions.

Program:

```
#include <stdio.h>
#define SIZE 10

int main(void) {
    int table[SIZE], keys[] = {23, 43, 13, 27};
    int n = 4, i, index, j, placed;

    for (i = 0; i < SIZE; i++) table[i] = -1;

    for (i = 0; i < n; i++) {
        index = keys[i] % SIZE;
        placed = 0;
        for (j = 0; j < SIZE; j++) {
            int pos = (index + j) % SIZE;
            if (table[pos] == -1) {
                table[pos] = keys[i];
                placed = 1;
                break;
            }
        }
        if (!placed)
            printf("Hash table is full; %d not inserted\n", keys[i]);
    }

    printf("Index  Value\n");
    for (i = 0; i < SIZE; i++)
        printf("%5d  %5d\n", i, table[i]);
    return 0;
}
```

Sample Output:

```
Index  Value
    0     -1
    1     -1
    2     -1
    3     23
    4     43
    5     13
    6     -1
    7     27
    8     -1
    9     -1
```

14(b). Hashing using Quadratic Probing

Aim: Insert keys into a hash table using quadratic probing to resolve collisions.

Program:

```
#include <stdio.h>
#define SIZE 11

int main(void) {
    int table[SIZE], keys[] = {22, 33, 44, 55};
    int n = 4, i, j, index, pos, placed;

    for (i = 0; i < SIZE; i++) table[i] = -1;

    for (i = 0; i < n; i++) {
        index = keys[i] % SIZE;
        placed = 0;
        for (j = 0; j < SIZE; j++) {
            pos = (index + j * j) % SIZE;
            if (table[pos] == -1) {
                table[pos] = keys[i];
                placed = 1;
                break;
            }
        }
        if (!placed)
            printf("Could not insert %d\n", keys[i]);
    }

    printf("Index  Value\n");
    for (i = 0; i < SIZE; i++)
        printf("%5d  %5d\n", i, table[i]);
    return 0;
}
```

Sample Output:

```
Index  Value
    0     22
    1     33
    2     -1
    3     -1
    4     44
    5     -1
    6     -1
    7     -1
    8     -1
    9     55
   10     -1
```